

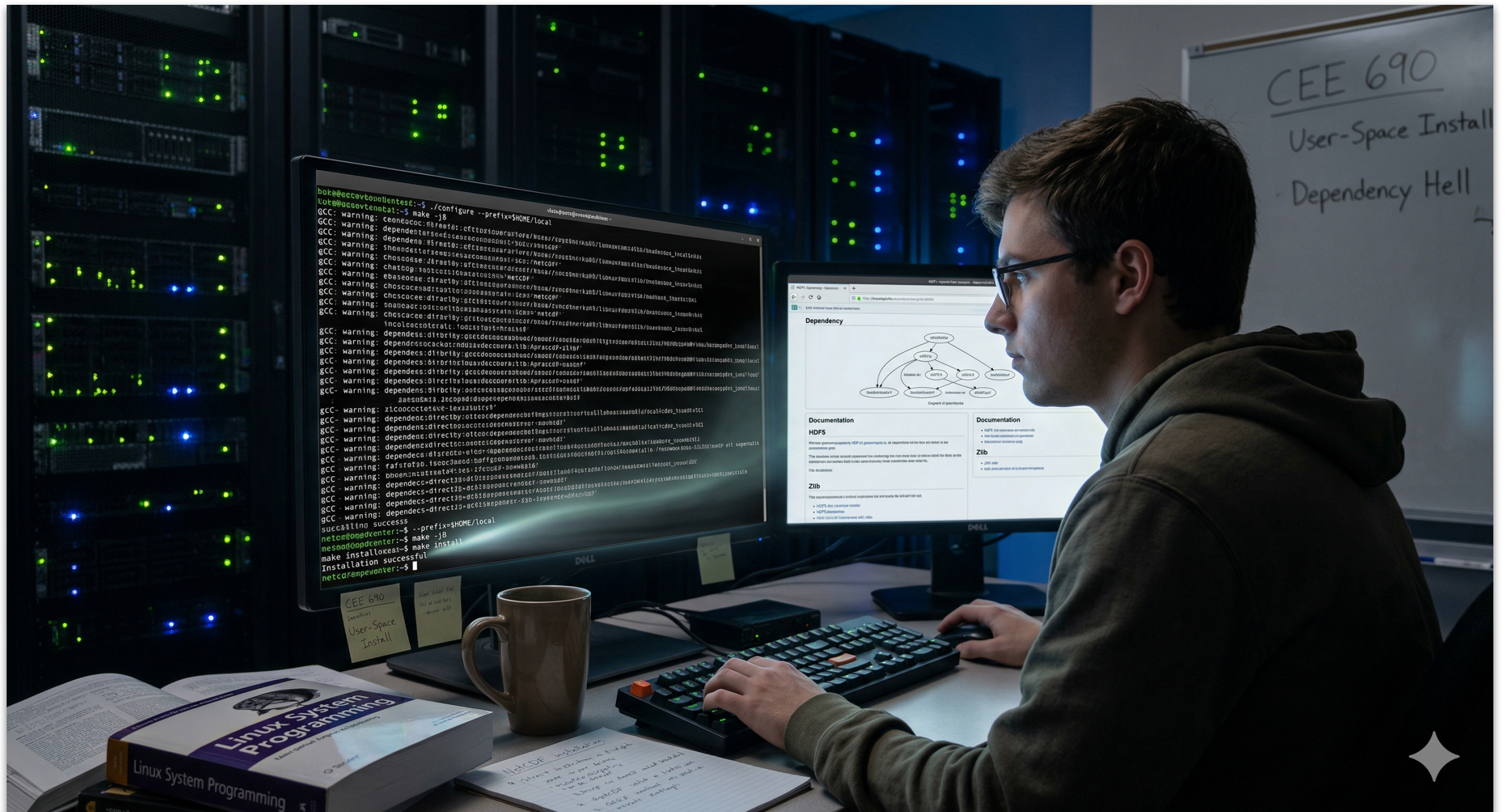
Lecture 17:

Compiling complex

software packages

CEE 690

Right of passage



Where are packages installed on an HPC system

```
[nc153 $ cd /
[nc153 $ ls
admin  boot  datacommons  fastrs  lib  media  opt  run  srv  usr  work
afs    cifs  dev          home   lib64  mnt  proc  sbin  sys  var
bin    core.d etc          hpc    lstor  nfs  root  scratch  tmp  vwork
```

/lib64 -> /usr/lib64

```
[nc153 $ cd lib64/
[nc153 $ ls
alsa-lib          libgdk-x11-2.0.so.0.2400.33  libnetsnmpagent.so.40  libtcmalloc_minimal.so.4.5.9
audit             libgd.so.3                  libnetsnmpagent.so.40.1.0  libtcmalloc.so.4
autofs            libgd.so.3.0.10             libnetsnmphelpers.so.40  libtcmalloc.so.4.5.9
bfd-plugins       libgettextlib-0.21.so        libnetsnmphelpers.so.40.1.0  libtdb.so.1
bpf               libgettextpo.so              libnetsnmphelpers.so.40.1.0  libtdb.so.1.4.13
ceph              libgettextpo.so.0            libnetsnmpmibs.so.40        libteamctl.so.0
cifs-utils        libgettextpo.so.0.5.7        libnetsnmpmibs.so.40.1.0  libteamctl.so.0.1.5
clang             libgettextpo.so.0.5.7        libnetsnmp.so.40           libteam.so.5
cmake             libgettextsrc-0.21.so        libnetsnmp.so.40.1.0       libteam.so.5.6.1
cracklib_dict.hwm libgexiv2.so.2               libnetsnmptrapd.so.40      libtermcap.so
cracklib_dict.pwd libgexiv2.so.2.14.3          libnetsnmptrapd.so.40.1.0  libtevent.so.0
                  libgfortran.so.5             libnet.so.1
```

/usr/include

```
[nc153 $ cd usr/include/
[nc153 $ ls
aio.h             crypt.h           envz.h            fts.h             gshadow.h         lzma               netash            panel.h           resolv.h          stdio_ext.h       tic.h              X11
aliases.h         ctype.h           errno.h           ftw.h             gsl                lzma.h            netatalk          paths.h           rpc               stdio.h           time.h            xcb
alloca.h          curl              errno.h           fwctl             iconv.h            malloc.h           netax25          pciaccess.h       sched.h           stdlib.h          ttyent.h          xen
a.out.h           cursesapp.h       error.h           gconv.h           ieee754.h          math.h            netdb.h          pcp               search.h          string.h           uchar.h           xf86drm.h
argp.h            cursesf.h         eti.h            gconv.h           ifaddrs.h          mcheck.h          netnet           poll.h            semaphore.h       strings.h          ucontext.h        xf86drmMode.h
argz.h            curses.h          etip.h           gelf.h            inttypes.h         memory.h          netipx           powercap.h        setjmp.h          syscalls.h        ulimit.h          zconf.h
ar.h              cursesm.h         execinfo.h       getopt.h          KHR                menu.h            netixcv          printf.h          setjmp.h          syscalls.h        unctrl.h          zdict.h
arpa              cursesw.h         fcntl.h          gettext-po.h      langinfo.h         misc              netpacket        proc_service.h    sgTTY.h           syscalls.h        unistd.h          zlib.h
asm               cursslk.h         features.h        GL                lastlog.h          libdrm            netrom           pthread.h         shadow.h           syslog.h           utime.h           zstd_errors.h
asm-generic       dirent.h          fenv.h           GLES              libelf.h           libm               netrose          pthread.h         signal.h           tar.h             utmp.h            zstd.h
assert.h           dlfcn.h          finclude         GLES2             libgen.h           libm               nfs              pty.h            slurm             termcap.h          utmpx.h           valgrind
autosprintf.h     dwarf.h           FlexLexer.h       GLES3             libintl.h          libm               nlist.h          pwd.h            sound             term_entry.h      values.h          video
bits              dwarf.h           fmtmsg.h          glob.h            libsync.h          libm               nl_types.h       python3.9         sqlite3ext.h      term.h            wait.h            wctype.h
byteswap.h         elf.h            fnmatch.h        glvnd             limits.h           libm               nss.h            readline          sqlite3.h         termios.h         wchar.h           wordexp.h
cpio.h            elfutils         fpu_control.h    gnu               link.h             libm               ncurses          regex.h           stab.h            thread_db.h        wctype.h
cpuidle.h          endian.h          ftab.h           grp.h             locale.h           libm               ncursesw         regexp.h          stdc-predef.h    threads.h          wordexp.h
```

No root access

- On your local machine you can just directly install to /lib /usr/include... (sudo access)
- On a cluster or HPC system you will not be able to do this and even if you ask for things to be installed it will either take a very long time or you will be denied
- “Modules” help but they still don’t give us control. We want to be able to install our own libraries/packages

Local directory

- Since we can't modify the root directory, we can install them and then have our environment variables “point” to where you have installed them.
- The most common approach is to create a “local” directory and then all the libraries, include, and binary files go inside there.

```
[nc153 $ cd $HOME
[nc153 $ ls
CEE690  Dev  local  miniconda3  ondemand  README  Software  spatialstats  tmp
[nc153 $ cd local/
[nc153 $ ls
bin  include  lib
```

(Detour) Fun fact... Local directories is exactly what conda environments are

- Instead of “\$HOME/local”, the environment directories have all the libraries, headers, etc...

```
[nc153 $ cd $HOME/miniconda3/envs
[nc153 $ ls
cee690
[nc153 $ cd cee690/
[nc153 $ ls
bin  conda-meta  etc  fonts  hdf5  include  lib  libexec  man  opt  sbin  share  ssl  var  x86_64-conda-linux-gnu
[nc153 $ cd lib
[nc153 $ ls
cairo                libformw.so          libcuc.so.78         lib
cmake                libformw.so.6        libcuc.so.78.1       lib
cups                 libformw.so.6.5      libiomp5.a           lib
dbus-1.0             libfreetype.so        libiomp5_db.so       lib
gdalplugins          libfreetype.so.6     libiomp5.so          lib
gdk-pixbuf-2.0       libfreetype.so.6.20.4 libiompstubs5.a      lib
girepository-1.0     libfreexl.a          libiompstubs5.so     lib
glib-2.0             libfreexl.so         libitm.so            lib
gnome-settings-daemon-3.0 libfreexl.so.1       libitm.so.1         lib
graphviz             libfreexl.so.1.2.0  libitm.so.1.0.0     lib
gtk-2.0              libfribidi.so        libjbig85.a          lib
gtk-3.0              libfribidi.so.0      libjbig.a            lib
```

- Every time you create a new conda environment you are making a contained directory (and that is why it works)

The search radius

- When you type a command or compile code, the node will only search the defaults (/usr/bin, /usr/lib, /usr/include)
- Setting an environment variable is telling the computer to widen the search radius to include my private directory (e.g., \$HOME/local)
- There are four big environments variable so consider

Environment Variable (EV) 1: PATH

- **What it does:** Tells the terminal where to look when you type a command (g++)
- **How to append?** `export PATH=$HOME/local/bin:$PATH`
- **Add :\$PATH at the end:** The last \$PATH is important as that create a new PATH that is a combination of the new directory as well as the already existing \$PATH variable

EV 2: CPPFLAGS/CPATH

- **What it does:** Tells the C/C++ preprocessor where to look for the .h and .hpp files
- **How to fix?** export CPPFLAGS="-I\$HOME/local/include"

EV 3: LD_FLAGS/LIBRARY_PATH

- **What it does:** Tells the linker where to look for the .a and .so files to use.
- **How to fix?** export LD_FLAGS="-L\$HOME/local/lib -L\$HOME/local/lib64"

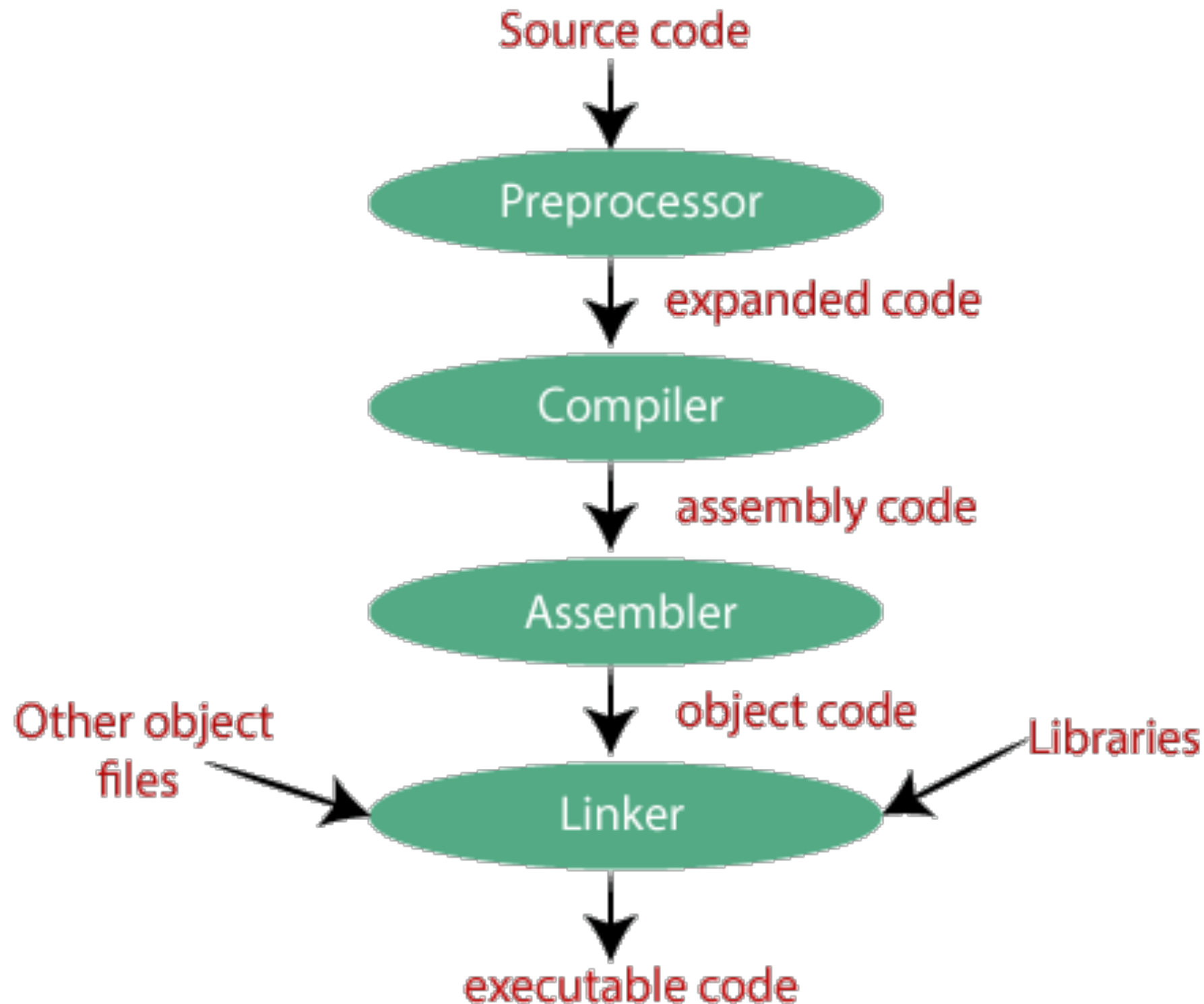
EV 4: LD_LIBRARY_PATH

- **What it does:** Tells the operating system where to find shared libraries (.so)
- **How to fix?** export LD_LIBRARY_PATH=\$HOME/local/lib:\$HOME/local/lib64:\$LD_LIBRARY_PATH
- **Runtime:** Unless we have added -Wl,-rpath at compilation we will need this at runtime so it can find the library

Add to .bashrc

```
# =====  
# CEE 690: Custom Software Environment  
# =====  
export MY_PREFIX=$HOME/local  
  
# 1. Add custom executables to the system path  
export PATH=$MY_PREFIX/bin:$PATH  
  
# 2. Add custom headers to the compiler search path  
export CPATH=$MY_PREFIX/include:$CPATH  
export CPPFLAGS="-I$MY_PREFIX/include"  
  
# 3. Add custom libraries to the Linker search path (Compile time)  
export LIBRARY_PATH=$MY_PREFIX/lib:$MY_PREFIX/lib64:$LIBRARY_PATH  
export LDFLAGS="-L$MY_PREFIX/lib -L$MY_PREFIX/lib64"  
  
# 4. Add custom libraries to the OS search path (Runtime)  
export LD_LIBRARY_PATH=$MY_PREFIX/lib:$MY_PREFIX/lib64:$LD_LIBRARY_PATH
```

Ready to compile software/library from source on cluster/HPC



Zlib: Compiling from source example

- We are going to work through compiling source with zlib as an example because it is pretty straightforward and it is a netCDF dependency.



- Zlib is a lossless data compression software library used in a lot of applications/tools.

Download source and unzip

- Move to a directory to download the source code

```
[nc153 $ ls  
CEE690 Dev local miniconda3 ondemand README Software spatialstats tmp  
[nc153 $ cd Software/
```

- Download it using wget (or whatever)

```
[nc153 $ wget https://zlib.net/current/zlib.tar.gz  
--2026-03-20 12:38:31-- https://zlib.net/current/zlib.tar.gz  
Resolving zlib.net (zlib.net)... 85.187.148.2  
Connecting to zlib.net (zlib.net)|85.187.148.2|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 1502830 (1.4M) [application/x-gzip]  
Saving to: 'zlib.tar.gz'  
  
zlib.tar.gz                               100%[=====]  
  
2026-03-20 12:38:31 (4.10 MB/s) - 'zlib.tar.gz' saved [1502830/1502830]
```

- Unzip and untar the tarball

```
[nc153 $ tar -xzf zlib.tar.gz  
zlib-1.3.2/  
zlib-1.3.2/zutil.h  
zlib-1.3.2/inftrees.h  
zlib-1.3.2/inflate.c  
zlib-1.3.2/CMakeLists.txt  
zlib-1.3.2/zlib.map  
zlib-1.3.2/zlib.pc.cmakein  
zlib-1.3.2/LICENSE  
zlib-1.3.2/zlib.3.pdf
```


Just run “make”, right? Not exactly

Default Makefile

```
all:
    -@echo "Please use ./configure first.  Thank you."

distclean:
    make -f Makefile.in distclean
```

```
[nc153 $ make
Please use ./configure first.  Thank you.
```

- We need to create a custom Makefile that takes into account the compiler, existing libraries, the settings, etc...
- Thankfully we don't need to do it ourselves.

./configure

- **Goal:** Create a Makefile customized for your specific machine
- **Options:** You can add user-specific flags to add to the makefile (e.g., `--prefix $HOME/local`)
- What it looks like: A lot of “**checking for ... yes**”
- **Where it fails:** When dependencies are missing

```
[nc153 $ ./configure --prefix=$HOME/local
Checking for gcc...
Checking for shared library support...
Building shared library libz.so.1.3.2 with gcc.
Checking for size_t... Yes.
Checking for off64_t... Yes.
Checking for fseeko... Yes.
Checking for strerror... Yes.
Checking for unistd.h... Yes.
Checking for stdarg.h... Yes.
Checking whether to use vs[n]printf() or s[n]printf()... using vs[n]printf().
Checking for vsnprintf() in stdio.h... Yes.
Checking for return value of vsnprintf()... Yes.
Checking for attribute(visibility) support... Yes.
Checking for s390x build ... No.
```

Created Makefile

```
CC=gcc
GCOV=gcov
LLVM_GCOV_FLAG=

CFLAGS=-O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN
#CFLAGS=-O -D_MAX_WBITS=14 -D_MAX_MEM_LEVEL=7
#CFLAGS=-g -DZLIB_DEBUG
#CFLAGS=-O3 -Wall -Wwrite-strings -Wpointer-arith -Wconversion \
#           -Wstrict-prototypes -Wmissing-prototypes

SFLAGS=-O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN
LDFLAGS=
TEST_LIBS=-L. libz.a
LD_SHARED=gcc -shared -Wl,-soname,libz.so.1,--version-script,zlib.map
CPP=
VGFMAFLAG=

STATIC_LIB=libz.a
SHARED_LIB=libz.so
SHARED_LIBV=libz.so.1.3.2
SHARED_LIBM=libz.so.1
LIBS=$(STATIC_LIB) $(SHARED_LIBV)

AR=ar
ARFLAGS=rc
RANLIB=ranlib
LD_CONFIG=ldconfig
LD_SHARED_LIBC=-lc
TAR=tar
SHELL=/bin/sh
EXE=

prefix =/hpc/home/nc153/local
exec_prefix =${prefix}
libdir =${exec_prefix}/lib
sharedlibdir =${libdir}
includedir =${prefix}/include
mandir =${prefix}/share/man
```

Compile the code (make)

- **Goal:** Compile “convert” the .cpp and .c source code files into binary .o object files and link them into libraries (.a/.so) or executables.
- **HPC compilation:** Use the -j flag (e.g., make -j 8). This tell the system to use 8 CPU cores at once. Problem?
- What it looks like: A lot of “**g++ -O2 -c ...**”
- **Where it fails:** Syntax error in the code or header file mentioned in the code is missing from the path

Compile the code (make)

```
[nc153 $ make
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -c -o example.o test/examp
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o Adler32.o Adler32.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o crc32.o crc32.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o deflate.o deflate.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o infback.o infback.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inffast.o inffast.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inflate.o inflate.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o inftrees.o inftrees.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o trees.o trees.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o zutil.o zutil.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o compress.o compress.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o uncompress.o uncompress.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzclose.o gzclose.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzlib.o gzlib.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzread.o gzread.c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -c -o gzwrite.o gzwrite.c
ar rc libz.a Adler32.o crc32.o deflate.o infback.o inffast.o inflate.o inftrees.o
.o gzread.o gzwrite.o
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o example example.o -L. libz
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -I. -c -o minigzip.o test/mini
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -o minigzip minigzip.o -L. li
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/Adler32.o ad
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/crc32.o crc3
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/deflate.o de
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/infback.o in
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/inffast.o in
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/inflate.o in
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/inftrees.o i
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/trees.o tree
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/zutil.o zuti
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/compress.o c
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/uncompress.o un
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/gzclose.o gz
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/gzlib.o gzli
gcc -O3 -fPIC -D_LARGEFILE64_SOURCE=1 -DHAVE_HIDDEN -DPIC -c -o objs/gzread.o gsr
```


The delivery (make install)

- **Goal:** Copy the finished binaries, headers, and libraries into the clean, organized structure defined in the `—prefix`
- **The Result:** It creates (or files) your `bin/`, `include/`, and `lib/` folders
- You can just run `make install` without first running `make`. You tend to avoid this so you can test the compiled code before you “deploy it”.

Place the compiled code (make install)

```
[nc153 $ make install
rm -f /hpc/home/nc153/local/lib/libz.a
cp libz.a /hpc/home/nc153/local/lib
chmod 644 /hpc/home/nc153/local/lib/libz.a
cp libz.so.1.3.2 /hpc/home/nc153/local/lib
chmod 755 /hpc/home/nc153/local/lib/libz.so.1.3.2
rm -f /hpc/home/nc153/local/share/man/man3/zlib.3
cp zlib.3 /hpc/home/nc153/local/share/man/man3
chmod 644 /hpc/home/nc153/local/share/man/man3/zlib.3
rm -f /hpc/home/nc153/local/lib/pkgconfig/zlib.pc
cp zlib.pc /hpc/home/nc153/local/lib/pkgconfig
chmod 644 /hpc/home/nc153/local/lib/pkgconfig/zlib.pc
rm -f /hpc/home/nc153/local/include/zlib.h /hpc/home/nc153/local/include/zconf.h
cp zlib.h zconf.h /hpc/home/nc153/local/include
chmod 644 /hpc/home/nc153/local/include/zlib.h /hpc/home/nc153/local/include/zconf.h
```

Compiling NetCDF

Phase 0: Set up the environment

```
# --- 0. THE BULLETPROOF ENVIRONMENT ---  
export MY_PREFIX=$HOME/local  
mkdir -p $MY_PREFIX  
  
export PATH=$MY_PREFIX/bin:$PATH  
export CPPFLAGS="-I$MY_PREFIX/include"  
# Point to BOTH lib and lib64 just in case  
export LDFLAGS="-L$MY_PREFIX/lib -L$MY_PREFIX/lib64"  
# Crucial for ./configure's internal tests  
export LD_LIBRARY_PATH=$MY_PREFIX/lib:$MY_PREFIX/lib64:$LD_LIBRARY_PATH  
export PKG_CONFIG_PATH=$MY_PREFIX/lib/pkgconfig:$MY_PREFIX/lib64/pkgconfig:
```


Phase 1: Zlib

```
tar -xzvf zlib-1.3.1.tar.gz
cd zlib-1.3.1
./configure --prefix=$MY_PREFIX
make -j4
make install
cd ..
```

Phase 2: HDF5

- HDF5 is the data engine under the hood of NetCDF

```
tar -xzvf hdf5-1.14.3.tar.gz
cd hdf5-1.14.3
./configure --prefix=$MY_PREFIX --with-zlib=$MY_PREFIX
make -j8
make install
cd ..
```

Phase 3: NetCDF-C

```
tar -xzvf netcdf-c-4.9.2.tar.gz  
cd netcdf-c-4.9.2  
./configure --prefix=$MY_PREFIX --disable-dap  
make -j4  
make install  
cd ..
```

Optional Phase: NetCDF C++ wrapper

```
tar -xzvf netcdf-cxx4-4.3.1.tar.gz  
cd netcdf-cxx4-4.3.1  
./configure --prefix=$MY_PREFIX  
make -j4  
make install  
cd ..
```


Does it work?

```
[(cee690) nc153 $ nc-config
```

```
Usage: nc-config [OPTION]
```

```
Available values for OPTION include:
```

--help	display this help message and exit
--all	display all options
--cc	C compiler
--cflags	pre-processor and compiler flags
--has-dap2	whether OPeNDAP (DAP2) is enabled in this build
--has-dap4	whether DAP4 is enabled in this build
--has-dap	same as --has-dap2 (Deprecated)
--has-nc2	whether NetCDF-2 API is enabled
--has-nc4	whether NetCDF-4/HDF-5 is enabled in this build
--has-hdf5	whether HDF5 is used in build (always the same as --has-nc4)
--has-hdf4	whether HDF4 was used in build
--has-logging	whether logging is enabled with --enable-logging.
--has-pnetcdf	whether PnetCDF was used in build
--has-szlib	whether szlib is included in build
--has-cdf5	whether cdf5 support is included in build
--has-parallel4	whether has parallel IO support via HDF5
--has-parallel	whether has parallel IO support via HDF5 or PnetCDF
--has-nczarr	whether nczarr enabled